

Học máy trong SLAM (Learning in SLAM)

PHỤ THUỘC A/06-bat-dinh-va-lan-truyen-sai-so (bất định — chương này lập luận rằng đây là chỗ học máy đóng góp được nhiều nhất và hiện đang đóng góp ít nhất); B/07-dac-trung-va-data-association (front-end học được); B/08-mo-hinh-phan-du (prior học được và ba điều kiện của nó); B/12-relocalization (trường hợp APR — một bài học phương pháp).

TÓM TẮT

Học máy đã thay đổi phần lớn thị giác máy tính, và câu hỏi tự nhiên là nó thay đổi SLAM tới đâu. Chương này lập luận rằng câu trả lời không đồng đều: có những chỗ học máy *đã* thắng rõ ràng và nên dùng ngay, có những chỗ nó chưa thắng và lý do là cấu trúc chứ không phải tạm thời, và có một chỗ nó *ên* đóng góp nhiều mà hiện đang đóng góp ít. Nội dung đi theo ba nhóm ấy. Nhóm một — front-end, prior độ sâu, place recognition — là nơi học máy thay một module có giao diện hẹp trong khi giữ nguyên back-end hình học, và đó là lý do nó thắng. Nhóm hai — SLAM đầu-cuối — là nơi nó chưa thắng, và mục 3 phân tích vì sao lý do là cấu trúc. Nhóm ba là bất định: đây là chỗ mà một mô hình học được *có thể* đóng góp nhiều nhất — học ra hiệp phương sai từ dữ liệu thay vì đoán — và là chỗ hiện có ít công trình nhất. Nếu chỉ nhớ một điều: *front-end học được cộng back-end hình học là hướng đúng*, và lý do không phải là thoả hiệp mà là phân công theo thế mạnh.

KIẾN THỨC CẦN CÓ TRƯỚC

- **front-end học được** — thay thế detector, descriptor hoặc matcher bằng một mạng học từ dữ liệu, giữ nguyên phần ước lượng hình học phía sau.
- **SLAM đầu-cuối (end-to-end)** — học một mạng ánh xạ trực tiếp từ chuỗi ảnh sang quỹ đạo, không có bước hình học tường minh nào.
- **BA khả vi (differentiable bundle adjustment)** — cài đặt bundle adjustment sao cho gradient chảy được qua nó, cho phép huấn luyện các thành phần phía trước bằng lan truyền ngược qua bộ giải.
- **foundation model 3D** — mô hình lớn huấn luyện trên dữ liệu đa dạng, cho ra cấu trúc 3D hoặc tương ứng trực tiếp từ ảnh mà không cần hiệu chuẩn hay tư thế cho trước.
- **hiệu chuẩn bất định (uncertainty calibration)** — tính chất mà một mô hình dự đoán kèm mức tin cậy *đúng* — khi nó nói “90% chắc” thì nó đúng 90% số lần. Phần lớn mạng nơ-ron không có tính chất này mà không được sửa.
- **ngoài phân bố (out-of-distribution)** — dữ liệu khác với dữ liệu huấn luyện. Mọi mô hình học được đều suy giảm ở đây, và vấn đề thật là chúng thường suy giảm *một cách im lặng*.
- **semantic SLAM** — đưa thông tin loại đối tượng vào bài toán ước lượng — landmark không chỉ là một điểm mà là “một góc của cái bàn”.

CÂU HỎI MỞ ĐẦU

- Q1. Học máy thắng rõ ở front-end nhưng chưa thắng ở back-end. Nêu một lý do *cấu trúc* cho sự bất đối xứng đó — không phải “vì chưa ai làm tốt”.
- Q2. DROID-SLAM dùng mạng cho luồng quang và một bộ giải BA ở giữa vòng huấn luyện. Nêu điều mà kiến trúc này giữ lại từ thể giới hình học, và vì sao việc giữ lại đó quan trọng.

- Q3. Một mô hình độ sâu cho ước lượng ở mọi pixel. Nếu điều nguy hiểm nhất về đầu ra của nó trong ngữ cảnh SLAM, và nói vì sao nó khác với một cảm biến độ sâu kém chính xác.
- Q4. Nếu bạn được chọn *một* chỗ để áp dụng học máy vào một hệ SLAM sản phẩm đang chạy, chọn chỗ nào và vì sao?
- Q5. Mô hình của bạn đạt kết quả tốt trên mọi benchmark công khai. Nếu ba câu hỏi phải trả lời trước khi đưa nó vào sản phẩm, và nói câu nào khó nhất.

1.1 Đặt vấn đề

Trong mười năm, học máy đã thay thế các phương pháp thủ công ở gần như mọi bài toán thị giác: phân loại, phát hiện, phân đoạn, sinh ảnh. Câu hỏi tự nhiên: vì sao SLAM vẫn chủ yếu là hình học?

Câu trả lời hời hợt là “vì chưa ai làm tốt”. Câu trả lời thật thú vị hơn, và nó nằm ở *bản chất của bài toán*.

Các bài toán mà học máy thắng có chung một đặc điểm: chúng là bài toán *nhận thức* — ánh xạ từ dữ liệu cảm giác sang một nhãn hoặc một mô tả, nơi quan hệ đúng là phức tạp, khó viết ra, và chỉ học được từ ví dụ. “Con mèo trông thế nào” không có công thức.

SLAM thì khác. Phần lõi của nó — cho các tương ứng, tìm tư thế và cấu trúc — là một bài toán *ước lượng* có một mô hình toán *đúng* và *đã biết*. Phép chiếu phối cảnh không phải thứ cần học từ dữ liệu; nó là hình học, đã đúng từ thế kỷ mười lăm. Một mạng học lại nó từ ví dụ sẽ học được một xấp xỉ tệ hơn, và nó chỉ đúng trong phạm vi dữ liệu huấn luyện — đúng như trường hợp APR ở B/12 mục 3B.

Nhưng SLAM *không chỉ* là bài toán ước lượng. Nó có một phần nhận thức rất lớn, và đó chính là nơi mọi khó khăn thực tế nằm: pixel nào tương ứng pixel nào, ảnh này có phải chỗ tôi đã đi qua không, vật kia có đang chuyển động không, độ sâu ở vùng trơn này khoảng bao nhiêu. Không câu nào trong số đó có công thức.

Từ đó ra một cách phân chia rõ ràng, và nó là luận điểm của chương: *học máy nên làm phần nhận thức, hình học nên làm phần ước lượng*. Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và nó là một lý do cấu trúc chứ không phải một quan sát tạm thời.

CẠM BÃY

Chương này chốt tại tháng 9/2026 và cùng với C/16 là phần hết hạn nhanh nhất của cây. Mọi đánh giá về mức trưởng thành dưới đây là *phán đoán của tôi*, không phải kết quả đo, và nên được rà lại sau sáu tháng.

Tôi cũng **không dẫn con số benchmark** cho bất kỳ so sánh nào trong chương. Lý do đã nêu ở B/07: các con số này phụ thuộc mạnh vào bộ dữ liệu và cách đo, và trong lĩnh vực học máy còn thêm một vấn đề — các baseline hình học thường được cài lại bởi nhóm đề xuất phương pháp học được, và theo *research-rules.md* mục 4 thì đó là một cờ đỏ cần cảnh giác.

Điều tôi đưa ra là phân tích về *cơ chế*: mỗi cách tiếp cận giữ lại gì từ hình học, bỏ đi gì, và điều đó có hệ quả gì. Phân tích ấy bền hơn con số.

TRỰC GIÁC

Hình dung việc đo một căn phòng bằng thước dây, và bạn có một người trợ lý. Có những việc người trợ lý làm tốt hơn bạn: *nhận ra* cái góc tường ở ảnh này chính là cái góc tường ở ảnh kia, dù ánh sáng khác và góc nhìn khác. Đó là việc nhận thức — nó đòi kinh nghiệm, và không có quy tắc nào viết ra được. Và có những việc bạn *không* nên nhờ trợ lý: cộng các số đo lại. Phép cộng có quy tắc đúng, đã biết, và không có lý do gì để đoán nó từ kinh nghiệm. Một người trợ lý giỏi “ước lượng tổng” vẫn thua một cái máy tính bỏ túi. Đó là toàn bộ sự phân công trong chương này. Học máy là người trợ lý có kinh nghiệm: giao cho nó việc nhận ra, việc phán đoán, việc điền vào chỗ trống. Hình học là cái máy tính: giao cho nó việc tính. Và có một chuyện thứ ba, quan trọng hơn cả hai. Người trợ lý tốt không chỉ nói “cái góc này khớp với cái góc kia” — anh ta nói thêm “tôi khá chắc” hoặc “cái này tôi đoán thôi”. Câu thứ hai đáng giá hơn nhiều so với vẻ ngoài, vì nó cho bạn biết nên tin điều gì. Phần lớn các mô hình học được hiện nay là những người trợ lý *không bao giờ nói câu thứ hai*. Họ trả lời mọi câu hỏi với cùng một giọng tự tin, kể cả những câu họ chưa từng gặp. Và đó là vấn đề lớn nhất khi đưa họ vào một hệ thống phải tự biết mình sai.

1.2 Nơi học máy đã thắng**A. Front-end**

Đây là chỗ rõ ràng nhất, và là câu trả lời cho câu hỏi mở đầu thứ tư.

SuperPoint [1], SuperGlue [2], LightGlue [3], LoFTR [4] — tất cả thay một module với *giao diện hẹp*: vào là hai ảnh, ra là một tập tương ứng. Mọi thứ phía sau không đổi.

Ba lý do vì sao đây là chỗ thắng, và chúng giải thích cả vì sao nó dễ áp dụng.

Một — đây thật sự là bài toán nhận thức. “Hai mảng ảnh này có phải cùng một điểm vật lý không” không có công thức; nó phụ thuộc vào việc bề mặt trông thế nào từ các góc khác nhau dưới các ánh sáng khác nhau, và đó là kiến thức thống kê về thế giới.

Hai — giao diện hẹp và kiểm được. Đầu ra là một tập tương ứng, và mọi tương ứng đều đi qua RANSAC và kiểm định hình học phía sau (B/07 mục 5). Nghĩa là một mô hình sai không phá hệ thống ngay — nó chỉ làm tỉ lệ inlier giảm, và tăng hình học bất được.

Ba — thay thế được từng phần. Đổi detector mà giữ nguyên matcher, hoặc ngược lại. Rủi ro triển khai thấp.

Cái giá đã nêu ở B/07 mục 2B và vẫn đúng: cần GPU, khó dự đoán khi ra ngoài phân bố, và mất tính tất định. Ba điều đó là lý do sản phẩm còn do dự chứ không phải vì chất lượng.

B. Prior độ sâu và place recognition

Hai chỗ thắng nữa với cùng cấu trúc lập luận.

Độ sâu học được (B/08 mục 4C, C/16 mục 2B) giải quyết được vùng không có vân, nơi hình học không có thông tin. Điều kiện dùng an toàn đã nêu: khởi tạo chứ không ràng buộc, và hiệp phương sai phải phản ánh sự thật.

Place recognition học được (B/11 mục 2C) bền hơn nhiều với thay đổi vẻ ngoài — mùa, ngày đêm. Đây rõ ràng là bài toán nhận thức, và các descriptor thủ công không có cách nào bắt được “cùng một chỗ, mùa khác”.

Điểm chung của cả ba chỗ thắng, và đáng nêu thành nguyên tắc: *học máy thắng ở nơi bài toán không có mô hình đúng, và nơi đầu ra được kiểm bởi một tầng hình học phía sau*. Hai điều kiện ấy đi cùng nhau.

1.3 Nơi nó chưa thắng, và vì sao

A. SLAM đầu-cuối

Ý tưởng: học một mạng ánh xạ từ chuỗi ảnh sang quỹ đạo, bỏ hết hình học.

Lập luận vì sao khó, và nó là lý do cấu trúc chứ không phải kỹ thuật.

Một — *nó phải học lại thứ đã biết*. Phép chiếu phối cảnh, hợp thành tư thế, ràng buộc epipolar — tất cả đều là toán đúng. Một mạng học chúng từ dữ liệu sẽ học một xấp xỉ, và xấp xỉ ấy chỉ đúng trong phạm vi dữ liệu huấn luyện. Trường hợp APR ở B/12 mục 3B là minh hoạ rõ nhất: mô hình nội suy giữa các tư thế huấn luyện thay vì học hình học.

Hai — *không có gauge*. SLAM có bảy bậc tự do gauge (A/05); một mạng đầu-cuối phải *chọn* một quy ước, và nó chọn quy ước của tập huấn luyện. Điều đó nghĩa là nó không tổng quát hoá sang môi trường có thang độ khác.

Ba — *không có hiệp phương sai có nghĩa*. Đầu ra là một tư thế, không phải một phân bố. Với A/06 là một chương của cây, đây là một thiếu sót nghiêm trọng cho ứng dụng.

Bốn — *không mở rộng được về thời gian*. Một mạng không có cơ chế nào tương đương với loop closure: nó không giữ một bản đồ có thể được sửa lại khi có thông tin mới.

B. Kiến trúc lai: DROID-SLAM

DROID-SLAM [5] là ví dụ tốt nhất về cách làm đúng, và nó là câu trả lời cho câu hỏi mở đầu thứ hai.

Kiến trúc: một mạng lập dự đoán luồng quang dày giữa các khung, và một lớp *bundle adjustment* lấy các luồng ấy làm phép đo rồi giải cho tư thế và độ sâu. Lớp BA nằm *trong* vòng lặp, và nó khả vi nên gradient chảy ngược qua nó để huấn luyện mạng.

Điều kiện trúc này giữ lại từ thế giới hình học — và đây là phần quan trọng: *ràng buộc hình học vẫn được áp đặt cứng*. Mạng không được phép đề xuất một quỹ đạo vi phạm phép chiếu; nó chỉ đề xuất *phép đo*, và hình học quyết định quỹ đạo. Nghĩa là mọi tính chất đúng đắn của BA — tính nhất quán hình học, khả năng dùng hiệp phương sai, khả năng thêm ràng buộc từ cảm biến khác — đều được giữ.

Vì sao việc giữ lại đó quan trọng: nó là thứ cho phép hệ thống *tổng quát hoá* ra ngoài dữ liệu huấn luyện theo hướng hình học. Mạng có thể chưa từng thấy một chuyển động như thế này, nhưng nếu nó cho luồng quang đúng thì BA cho tư thế đúng — vì BA không cần học.

Đây là lập luận chính của chương, phát biểu ở dạng cụ thể: *đặt học máy ở chỗ sinh ra phép đo, đặt hình học ở chỗ biến phép đo thành trạng thái*.

C. Foundation model 3D

DUST3R [6], MAST3R [7], VGGT [8] đi theo một hướng khác và đáng chú ý: cho hai hoặc nhiều ảnh bất kỳ, trực tiếp cho ra cấu trúc 3D và tương ứng, *không cần* hiệu chuẩn hay tư thế cho trước.

Đây là một thay đổi thật về cách nghĩ, vì nó xoá bỏ giả định mà cả A/01 và A/02 dựa vào — rằng ta biết tham số nội. Với ảnh thu thập từ nguồn không rõ, đó là một lợi ích lớn.

Đánh giá của tôi tại 9/2026: rất hứa hẹn cho SfM và cho tái tạo từ ảnh không hiệu chuẩn; *chưa* là lựa chọn cho SLAM thời gian thực trên thiết bị, vì chi phí tính toán và vì thiếu cơ chế cho hoạt động dài hạn (bản đồ, loop closure, quản lý). Cách dùng thực dụng nhất hiện nay có lẽ là làm *bộ khởi tạo* và làm công cụ dựng bản đồ ngoại tuyến, rồi một hệ hình học nhẹ chạy localization trên bản đồ đó. (*Đây là phán đoán của tôi, chưa đối chiếu với một khảo sát có hệ thống.*)

1.4 Nơi nó nên đóng góp mà chưa

Đây là mục mà tôi cho là có giá trị cao nhất của chương, và là câu trả lời cho câu hỏi mở đầu thứ ba.

A. Bất định là chỗ trống

A/06 lập luận rằng hiệp phương sai đáng tin là thứ SLAM thiếu một cách hệ thống, và rằng bốn nguồn của bệnh quá tự tin đều khó chữa bằng phân tích.

Đây chính là loại bài toán mà học máy *nên* giải: ước lượng một phân bố từ dữ liệu, khi ta không viết được mô hình giải tích cho nó. “Khớp điểm này đáng tin tới đâu, cho loại cảnh này, dưới ánh sáng này” không có công thức — nhưng có rất nhiều dữ liệu.

Vậy mà số công trình về bất định học được trong SLAM ít hơn nhiều so với số công trình về front-end học được. (*Đây là ấn tượng của tôi từ việc đọc rải rác, không phải kết quả của một khảo sát có hệ thống — và làm khảo sát ấy là một mục trong 99-chưa-biết.md.*)

Tôi cho rằng có hai lý do. *Một*: khó đo. Để biết một ước lượng bất định có đúng không, cần chân trị và cần thống kê — tức cần NEES (A/06 mục 4A), và đó là một phép đo mà cộng đồng ít dùng. Không có chỉ số dễ đo thì không có tiến bộ dễ công bố. *Hai*: ít hấp dẫn. Một bài báo cải thiện ATE dễ đăng hơn một bài báo cải thiện tính trung thực của hiệp phương sai, dù cái thứ hai có giá trị ứng dụng cao hơn.

B. Vấn đề hiệu chuẩn bất định

Có một khó khăn thật, không chỉ là thiếu quan tâm, và nó đáng nêu rõ.

Mạng nơ-ron nổi tiếng là *không được hiệu chuẩn* về mặt xác suất: khi một mô hình phân loại nói “99% chắc”, nó thường đúng ít hơn 99% số lần. Với SLAM, hệ quả nghiêm trọng hơn so với với phân loại, vì hiệp phương sai không chỉ được *đọc* mà được *dùng làm trọng số* — một hiệp phương sai quá nhỏ kéo cả nghiệm đi (B/08 khối *Cạm bẫy*).

Và vấn đề gay gắt nhất: bất định phải đúng *ngoài phân bố huấn luyện*, vì đó chính là lúc ta cần nó nhất. Một mô hình biết mình không chắc khi gặp cảnh lạ là đúng thứ ta muốn — và đó cũng là thứ khó nhất để huấn luyện, vì theo định nghĩa ta không có dữ liệu huấn luyện cho “cảnh lạ”.

Đây là câu trả lời cho câu hỏi mở đầu thứ ba: điều nguy hiểm nhất về đầu ra của một mô hình độ sâu không phải nó sai, mà là nó *sai với vẻ chắc chắn như nhau ở mọi pixel*. Một cảm biến độ sâu kém chính xác vẫn tốt hơn, vì sai số của nó có cấu trúc đã biết — nó tệ ở xa, tệ trên bề mặt bóng, và ta mô hình hoá được điều đó. Mô hình học được không cho ta cấu trúc ấy.

C. Semantic SLAM: một hướng khác

Đưa thông tin loại đối tượng vào bài toán: landmark không chỉ là một điểm mà là “một góc của cái bàn”. CubeSLAM [9] và QuadricSLAM [10] biểu diễn đối tượng bằng các hình đơn giản và đưa chúng vào đồ thị như landmark cấp cao.

Ba lợi ích tiềm năng. *Data association tốt hơn*: hai điểm thuộc hai đối tượng khác loại chắc chắn không phải cùng một điểm — một ràng buộc mạnh và miễn phí. *Xử lý vật động*: biết cái gì là xe thì biết cái gì có thể di chuyển (D/19). *Bản đồ ổn định hơn*: các đối tượng cố định (tường, cột) đáng tin hơn các đối tượng di động (ghế), và bản đồ nên phản ánh điều đó — một hướng tấn công thật vào bài toán lifelong ở C/13 mục 4C.

Đánh giá tại 9/2026: lợi ích thứ ba là lợi ích lớn nhất và ít được khai thác nhất. Phần lớn công trình semantic SLAM tập trung vào việc dựng bản đồ có nhãn, trong khi giá trị thực dụng cao hơn nằm ở việc dùng nhãn để *quyết định tin cái gì*. (Phán đoán của tôi.)

CẦN NHỚ

Học máy làm phần nhận thức, hình học làm phần ước lượng — và ranh giới giữa hai phần là chỗ *phép đo* được sinh ra.

Cụ thể: đặt mô hình học được ở chỗ trả lời các câu không có công thức — pixel nào khớp pixel nào, ảnh này có phải chỗ cũ không, độ sâu ở vùng trơn này khoảng bao nhiêu, điểm này có nằm trên vật động không. Đặt hình học ở chỗ biến các câu trả lời ấy thành trạng thái.

Ba lý do vì sao ranh giới này đúng, không phải một thoả hiệp. *Một*: phần ước lượng có mô hình đúng và đã biết, nên học nó từ dữ liệu chỉ cho một xấp xỉ tệ hơn. *Hai*: giữ hình học nghĩa là giữ được hiệp phương sai, gauge, khả năng thêm cảm biến, và loop closure — bốn thứ mà một mạng đầu-cuối mất hết. *Ba*: đầu ra của mô hình học được đi qua một tầng kiểm định hình học, nên một mô hình sai làm giảm chất lượng chứ không phá hệ thống.

Và một điều kiện bắt buộc mà hiện ít được đáp ứng: **mô hình phải nói được nó chắc tới đâu**, và con số ấy phải được hiệu chuẩn trên dữ liệu thật. Không có nó, phép đo do mô hình sinh ra không đưa vào đồ thị một cách hợp lệ được — xem B/08 mục 4C.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
SuperPoint + LightGlue [3]	Front-end học được, giao diện hẹp	Mục 2A — chỗ dễ áp dụng nhất và lợi ích rõ nhất
DROID-SLAM [5]	Luồng học được + lớp BA khả vi trong vòng lặp	Mục 3B — kiến trúc lai làm đúng; đọc để thấy ranh giới đặt ở đâu
DPVO [11]	Phiên bản nhẹ hơn nhiều của cùng ý tưởng	Cho thấy chi phí có thể giảm mạnh mà giữ kiến trúc
MASt3R [7], VGGT [8]	Foundation model 3D không cần hiệu chuẩn	Mục 3C — thay đổi cách nghĩ về SfM; chốt 9/2026
Theseus [12]	Thư viện tối ưu phi tuyến khả vi	Công cụ để xây kiến trúc lai; hạ tầng cho mục 3B
DynaSLAM [13]	Phân đoạn ngữ nghĩa để loại vật động	Ứng dụng thực dụng nhất của semantic; xem D/19

MINI PROJECT SLAM · Thay front-end bằng bản học được, rồi đo cái mà benchmark không đo · 3 ngày

Mục tiêu đo lợi ích thật của front-end học được trên dữ liệu của bạn, và đo cái giá mà bảng so sánh không nhắc — hành vi ngoài phân bố và tính bất định.

Dữ liệu một chuỗi chuẩn (EuRoC), cộng *ba* chuỗi tự thu cố ý ngoài phân bố: một chuỗi ánh sáng rất kém, một chuỗi trong môi trường khác hẳn dữ liệu huấn luyện phổ biến (nhà kho công nghiệp, ngoài trời ban đêm), và một chuỗi có chuyển động rất nhanh.

Các bước	chạy cùng một hệ SLAM với hai front-end — ORB và SuperPoint+LightGlue — trên cả bốn chuỗi; đo ATE, tỉ lệ tracking, thời gian mỗi khung; rồi chạy <i>mỗi cấu hình hai lần trên cùng dữ liệu</i> và kiểm xem kết quả có bit-exact không.
Xong khi	bạn có một bảng hai front-end nhân bốn chuỗi với bốn cột. Ba điều cần quan sát: (1) trên chuỗi chuẩn, học được thắng hoặc hoà — đúng như mong đợi; (2) trên các chuỗi ngoài phân bố, xem nó thắng ở đâu và <i>thua ở đâu</i> — đây là phần benchmark không cho bạn; (3) kiểm tính tất định — nếu hai lần chạy cho kết quả khác nhau, bạn vừa phát hiện một rào cản triển khai thật mà không bài báo nào nhắc tới (D/20).
Nối với	D/19-ben-vung-trong-the-gioi-thuc cho danh mục điều kiện khó đầy đủ hơn; D/21-tu-prototype-len-mass-production bàn về chi phí cập nhật mô hình và tái kiểm định.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Lý do cấu trúc: phần lõi của SLAM là bài toán *ước lượng* có một mô hình toán *đúng và đã biết* — phép chiếu phối cảnh không phải thứ cần học từ dữ liệu. Một mạng học lại nó sẽ học một xấp xỉ chỉ đúng trong phạm vi dữ liệu huấn luyện, như trường hợp APR ở B/12. Front-end thì ngược lại: “hai mảng ảnh này có phải cùng một điểm không” là bài toán *nhận thức*, không có công thức, phụ thuộc vào kiến thức thống kê về việc bề mặt trông thế nào từ các góc và ánh sáng khác nhau. Học máy thắng ở nơi không có mô hình đúng, và thua ở nơi có.
- A2. Nó giữ lại việc *áp đặt cứng các ràng buộc hình học*: mạng chỉ được đề xuất *phép đo* (luồng quang), còn lớp BA quyết định quỹ đạo. Mạng không được phép đề xuất một quỹ đạo vi phạm phép chiếu. Việc giữ lại đó quan trọng vì nó cho phép hệ tổng quát hoá theo hướng hình học ra ngoài dữ liệu huấn luyện — mạng có thể chưa từng thấy chuyển động này, nhưng nếu nó cho luồng đúng thì BA cho tư thế đúng, vì BA không cần học. Nó cũng giữ được hiệp phương sai, gauge, khả năng thêm cảm biến và loop closure — bốn thứ mà mạng đầu-cuối mất hết.
- A3. Điều nguy hiểm nhất là nó *sai với vẻ chắc chắn như nhau ở mọi pixel* — kể cả ở những pixel mà hình học không nói gì và mô hình chỉ đang nội suy từ tiên nghiệm. Nó khác với một cảm biến độ sâu kém chính xác ở chỗ sai số của cảm biến có *cấu trúc đã biết*: nó tệ ở xa theo z^2 , tệ trên bề mặt bóng, và ta mô hình hoá được điều đó rồi đặt trọng số tương ứng. Mô hình học được không cho ta cấu trúc ấy, nên ta không biết đặt trọng số thế nào — và với ứng dụng an toàn, một bản đồ trộn giữa phép đo và phỏng đoán mà không phân biệt được hai phần là nguy hiểm.
- A4. Front-end — cụ thể là matcher. Ba lý do: (1) đây thật sự là bài toán nhận thức nên lợi ích lớn; (2) giao diện hẹp — vào hai ảnh, ra một tập tương ứng — nên thay được mà không đụng gì khác; (3) đầu ra đi qua RANSAC và kiểm định hình học, nên một mô hình kém làm giảm tỉ lệ inlier chứ không phá hệ thống. Rủi ro triển khai thấp nhất trên mỗi đơn vị lợi ích. Điều kiện: phải có ngân sách GPU, và phải chấp nhận mất tính tất định — hai cái giá thuộc về D/20 chứ không phải về chất lượng.
- A5. Ba câu: (1) *nó hành xử thế nào ngoài phân bố huấn luyện*, và nó có *biết* mình đang ở ngoài không? (2) *nó có tất định không* — cùng đầu vào cho cùng đầu ra bit-exact, điều kiện để tái hiện một lỗi từ hiện trường? (3) *chi phí cập nhật là bao nhiêu* — đổi mô hình đòi tái kiểm định những gì? Câu (1) khó nhất, vì theo định nghĩa ta không có dữ liệu để kiểm “cảnh lạ”, và vì việc huấn luyện một mô hình *biết* mình không chắc khi gặp cảnh lạ là bài toán chưa giải. Điểm số tốt trên benchmark công khai không trả lời câu nào trong ba câu ấy.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đo một căn phòng bằng thước dây và có một người trợ lý giàu kinh nghiệm. Có việc anh ta làm giỏi hơn bạn: nhận ra cái góc tường trong tấm ảnh này chính là cái góc tường trong tấm ảnh kia, dù chụp lúc khác, sáng khác, đứng ở chỗ khác. Việc đó không có quy tắc — nó là kinh nghiệm. Và có việc bạn không nên nhờ anh ta: cộng các con số lại. Phép cộng có quy tắc đúng, đã biết từ lâu. Một người “ước lượng tổng” giỏi đến mấy vẫn thua cái máy tính bỏ túi.

Đó là toàn bộ cách phân công. Giao việc nhận ra cho người có kinh nghiệm; giao việc tính cho cái máy tính. Và ranh giới giữa hai việc nằm đúng ở chỗ con số được sinh ra: người trợ lý đọc ra các con số, cái máy tính xử lý chúng.

Nhưng có một chuyện thứ ba, và nó quan trọng hơn cả hai chuyện trên.

Một người trợ lý tốt không chỉ nói “góc này khớp góc kia”. Anh ta nói thêm: “cái này tôi chắc” hoặc “cái này tôi đoán thôi, ánh sáng tối quá”. Câu thứ hai đáng giá hơn nhiều so với vẻ ngoài, vì nó cho bạn biết nên tin đến đâu — và nên đi kiểm lại chỗ nào.

Phần lớn các mô hình học máy hiện nay là những người trợ lý *không bao giờ nói câu thứ hai*. Họ trả lời mọi câu với cùng một giọng chắc chắn, kể cả những câu họ chưa từng gặp bao giờ. Bạn đưa cho họ một tấm ảnh chụp trong một nhà kho mà họ chưa từng thấy loại nào tương tự, và họ vẫn trả lời rành rọt.

Đó là vấn đề lớn nhất khi đưa họ vào một cái máy phải tự biết mình sai. Và điều trở trêu là: dạy máy *nói không chắc* đúng chỗ lại là thứ mà máy học lẽ ra nên giỏi — ước lượng mức tin cậy từ dữ liệu là đúng loại việc của nó. Chỉ là chưa mấy ai làm.

Chỗ không rút gọn được: chỗ không rút gọn được là việc dạy một mô hình biết mình đang gặp thứ chưa từng thấy. Theo định nghĩa, bạn không có dữ liệu huấn luyện cho “những thứ chưa từng thấy”. Mọi cách tiếp cận hiện có đều là cách vòng quanh vấn đề ấy, và không cách nào thỏa đáng.

Một câu: *Giao việc nhận ra cho cái học được, giao việc tính cho cái đã biết — và đòi cái học được phải nói rõ khi nó chỉ đang đoán.*

1.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *học máy nên làm nhận thức, hình học nên làm ước lượng* — bị bác bỏ nếu một hệ đầu-cuối thuần túy vượt một hệ lai trên các chuỗi *ngoài phân bố huấn luyện*, với cùng ngân sách tính toán. Điều kiện “ngoài phân bố” là chỗ phải kiểm nghiệm khác, vì trong phân bố thì một mạng đủ lớn có thể thắng mà không chứng minh gì về khả năng tổng quát hoá. Tôi cho là mệnh đề vững, nhưng ranh giới *đang dịch* — các foundation model ở mục 3C hấp thụ ngày càng nhiều phần hình học vào mô hình, và tôi không chắc phát biểu này còn nguyên vẹn sau vài năm.

Mệnh đề thứ hai — *bất định học được là chỗ trống* — dựa trên ấn tượng đọc rải rác chứ không trên khảo sát, và tôi đã ghi rõ điều đó ở mục 4A. Nó bị bác bỏ đơn giản bằng cách chỉ ra một nhánh công trình đáng kể mà tôi bỏ sót. Đây là mục có rủi ro sai cao nhất trong chương, và việc làm khảo sát cho nó nằm trong 99-chua-biet.md.

Mệnh đề thứ ba — *mạng nơ-ron không được hiệu chuẩn về xác suất* — là một kết quả đã biết rộng rãi trong cộng đồng học máy, nhưng tôi *không* dẫn được một nguồn cụ thể trong ngữ cảnh SLAM, và tôi không biết mức độ nghiêm trọng của nó cho các mô hình độ sâu và luồng quang cụ thể đang dùng. Đây là một khoản nợ kiểm chứng.

Mệnh đề thứ tư — đánh giá về mức trưởng thành của foundation model tại 9/2026 — là phán đoán và sẽ hết hạn. Rà lại sau sáu tháng.

1.6 Kết nối

Nói lên trên. B/07-dac-trung-va-data-association là chỗ front-end học được được bàn ở mức chi tiết, cùng cái giá của nó. B/08-mo-hinh-phan-du mục 4C cho ba điều kiện đưa một prior học được vào đồ thị, và chúng áp dụng nguyên vẹn cho mọi nguồn học được. B/12-relocalization mục 3B cho trường hợp APR, và bài học phương pháp ở đó — so với một đường cơ sở tầm thường — nên áp dụng cho mọi khẳng định trong chương này. A/06-bat-dinh-va-lan-truyen-sai-so là chương mà mục 4 nối vào: bất định là chỗ trống, và lý do nó là chỗ trống nằm ở chỗ nó khó đo.

Nổi ngang. C/16-dense-mapping-va-bieu-dien chia sẻ đặc điểm hết hạn nhanh, và hai chương nên được rà cùng nhau. C/18-danh-gia-va-phuong-phap-luan cho công cụ để đánh giá một thành phần học được một cách có kỷ luật — đặc biệt là yêu cầu về baseline và về đánh giá ở đuôi phân bố.

Nổi xuống dưới. D/19-ben-vung-trong-the-gioi-thuc là chỗ semantic segmentation có ứng dụng thực dụng nhất: loại vật động. D/20-toi-uu-hieu-nang-va-he-thong bàn về chi phí tính toán và về tính tất định — hai cái giá lớn nhất của front-end học được trong sản phẩm. D/21-tu-prototype-len-mass-production bàn về chi phí cập nhật mô hình, tái kiểm định, và OTA cho mô hình — những thứ không tồn tại với một thuật toán thủ công. D/22-huong-nghien-cuu-con-trong lay “front-end học được + back-end hình học với bất định được truyền đúng” làm hướng số năm, và lý do nằm trọn trong mục 4 của chương này.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu lý do *cấu trúc* vì sao học máy thắng ở front-end mà chưa thắng ở back-end.
2. Ba điều kiện chung của các chỗ mà học máy đã thắng là gì, và vì sao chúng đi cùng nhau?
3. DROID-SLAM giữ lại gì từ hình học, và điều đó cho hệ thống khả năng gì mà một mạng đầu-cuối không có?
4. Nêu bốn thứ mà một hệ SLAM đầu-cuối mất so với một hệ hình học.
5. Vì sao bất định là chỗ mà học máy *ên* đóng góp nhiều nhất, và nêu hai lý do vì sao hiện nó đóng góp ít?
6. Vì sao việc huấn luyện một mô hình “biết mình không chắc khi gặp cảnh lạ” là bài toán khó về mặt nguyên tắc?
7. Nêu ba câu hỏi phải trả lời trước khi đưa một mô hình học được vào sản phẩm, và nói vì sao benchmark không trả lời câu nào.

Nguồn

- [1] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [2] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [3] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [4] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [5] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin A, 2021.
- [6] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [7] V. Leroy, Y. Cabon, and J. Revaud, “Grounding Image Matching in 3D with MAST3R,” in *European Conference on Computer Vision (ECCV)*, mức tin B, 2024.
- [8] J. Wang, M. Chen, N. Karaev, et al., “VGGT: Visual Geometry Grounded Transformer,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin C; rất mới, chưa có tái lập độc lập rộng rãi, 2025.
- [9] S. Yang and S. Scherer, “CubeSLAM: Monocular 3-D Object SLAM,” in *IEEE Transactions on Robotics*, mức tin B, 2019.
- [10] L. Nicholson, M. Milford, and N. Sünderhauf, “QuadricSLAM: Dual Quadrics from Object Detections as Landmarks in Object-Oriented SLAM,” in *IEEE Robotics and Automation Letters*, mức tin B, 2019.
- [11] Z. Teed, L. Lipson, and J. Deng, “Deep Patch Visual Odometry,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2023.
- [12] L. Pineda, T. Fan, M. Monge, et al., “Theseus: A Library for Differentiable Nonlinear Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2022.
- [13] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.